

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÀI TẬP LỚN
**PHÁT TRIỂN ỨNG DỤNG INTERNET OF
THING (CO3037)**

Internet of Things Project Assignment

Giảng viên hướng dẫn: Lê Trọng Nhân
HK252

STT	Họ và tên	MSSV	Lớp	Ghi chú
1	Hồ Đăng Khoa	211588	L01	Nhóm trưởng
2	Hoàng Sỹ Xuân Sơn	2212937	L01	
3	Vũ Đức Lâm	2212466	L01	
4	Nguyễn Minh Hưng	2211367	L01	

Thành phố Hồ Chí Minh, tháng 5 năm 2026

Danh sách thành viên

STT	Họ và tên	MSSV	Nhiệm vụ	Đóng góp
1	Hồ Đăng Khoa	2211588		100%
2	Hoàng Sỹ Xuân Sơn	2212937		100%
3	Vũ Đức Lâm	2212466		100%
4	Nguyễn Minh Hưng	2211367		100 %



Mục lục

1 Giới thiệu

1.1 Lý do chọn đề tài

Trong sự phát triển không ngừng của Internet vạn vật (IoT), việc xây dựng các hệ thống nhúng đòi hỏi khả năng xử lý nhiều tác vụ đồng thời với độ trễ thấp và độ chính xác cao. Để đáp ứng yêu cầu khắt khe này, Hệ điều hành thời gian thực (RTOS) trở thành một công cụ không thể thiếu đối với các kỹ sư và lập trình viên. Việc ứng dụng RTOS giúp tối ưu hóa tài nguyên phần cứng, đồng thời quản lý hiệu quả các luồng dữ liệu liên tục từ cảm biến.

Xuất phát từ nhu cầu nắm bắt các kỹ thuật lập trình nhúng nâng cao, nhóm đã quyết định thực hiện đề tài dựa trên việc sửa đổi và mở rộng dự án nền tảng YoloUNO_PlatformIO - RTOS_Project. Đề tài mang lại cơ hội thực hành trực tiếp các kỹ thuật đồng bộ hóa tác vụ bằng tín hiệu (signals) và semaphore, loại bỏ hoàn toàn việc sử dụng các biến toàn cục thiếu an toàn. Đồng thời, dự án còn tích hợp các công nghệ hiện đại như Trí tuệ nhân tạo tại biên (TinyML) và giao tiếp đám mây (Cloud Server), tạo tiền đề vững chắc cho việc phát triển các sản phẩm IoT phức tạp trong thực tế.

1.2 Bối cảnh ứng dụng thực tế

Các hệ thống nhúng hiện đại không chỉ dừng lại ở việc thu thập dữ liệu đơn thuần mà đang tiến tới khả năng "tự suy nghĩ" và phân tích dữ liệu ngay tại nơi thu thập (Edge Computing). Trong bối cảnh đó, sự kết hợp giữa RTOS và TinyML mở ra khả năng tạo ra các thiết bị thông minh có thể nhận diện môi trường, dự đoán sự cố và phản hồi tức thì mà không cần phụ thuộc hoàn toàn vào máy chủ trung tâm.

Mô hình dự án này phản ánh trực tiếp cấu trúc của một hệ thống IoT công nghiệp cỡ nhỏ: từ lớp cảm nhận (thu thập nhiệt độ, độ ẩm), lớp xử lý cục bộ (giao diện Web AP, xử lý TinyML, cảnh báo qua LCD/LED), cho đến lớp mạng và đám mây (truyền dữ liệu lên nền tảng CoreIOT). Những kiến thức và kỹ năng từ dự án này có thể áp dụng trực tiếp vào việc thiết kế các hệ thống nhà thông minh (Smart Home), trạm quan trắc nông nghiệp, hoặc thiết bị y tế đeo tay.

1.3 Mục tiêu của hệ thống

Mục tiêu chính của đề tài là xây dựng một hệ thống giám sát và điều khiển thông minh trên vi điều khiển ESP32 S3, đảm bảo mã nguồn mới phải có chức năng khác biệt ít nhất 30% so với mã nguồn gốc. Để đạt được điều này, hệ thống cần hoàn thành các mục tiêu cụ thể sau:

- **Cải tiến kỹ thuật nhúng:** Quản lý trạng thái nhấp nháy của đèn LED theo nhiệt độ và cấu hình màu sắc của LED NeoPixel dựa trên độ ẩm (ít nhất 3 mức độ khác nhau) thông qua semaphore và tín hiệu đồng bộ.
- **Quản lý hiển thị:** Sử dụng semaphore để quản lý tài nguyên, hiển thị thông số và ít nhất 3 trạng thái cảnh báo trên màn hình LCD mà không dùng biến toàn cục.
- **Giao diện tương tác:** Thiết lập ESP32 S3 làm điểm truy cập (Access Point) và thiết kế một máy chủ web trực quan cho phép người dùng điều khiển ít nhất hai thiết bị ngoại vi.
- **Triển khai TinyML:** Xây dựng bộ dữ liệu, huấn luyện, chạy mô hình TinyML trên phần cứng và tiến hành đo lường độ chính xác.
- **Tích hợp đám mây:** Cấu hình ESP32 S3 ở chế độ Trạm (STA) để xuất bản thành công dữ liệu cảm biến lên máy chủ đám mây CoreIOT bằng mã xác thực hợp lệ.

1.4 Phạm vi và giới hạn

Dự án được triển khai toàn bộ trên nền tảng vi điều khiển ESP32 S3 bằng ngôn ngữ C/C++ thông qua plugin PlatformIO. Trong giới hạn của đề tài, cấu hình phần cứng của bo mạch được cố định ở chế độ 301B9 và 812H6.

Về mặt dữ liệu và kết nối, hệ thống giới hạn việc thu thập các thông số cơ bản (nhiệt độ, độ ẩm) và truyền dữ liệu thông qua giao thức Wi-Fi hiện có tại phòng thí nghiệm. Quá trình huấn luyện mô hình TinyML sử dụng tập dữ liệu có quy mô nhỏ gọn, phù



hợp với dung lượng bộ nhớ của ESP32 S3, và nền tảng CoreIOT được sử dụng như giải pháp lưu trữ đám mây duy nhất thay vì triển khai một máy chủ (Server) độc lập. Việc kiểm thử hệ thống được thực hiện trong môi trường mô phỏng (phòng Lab), tập trung vào tính logic của thuật toán quản lý RTOS thay vì tối ưu hóa cho điều kiện hoạt động khắc nghiệt ngoài trời.

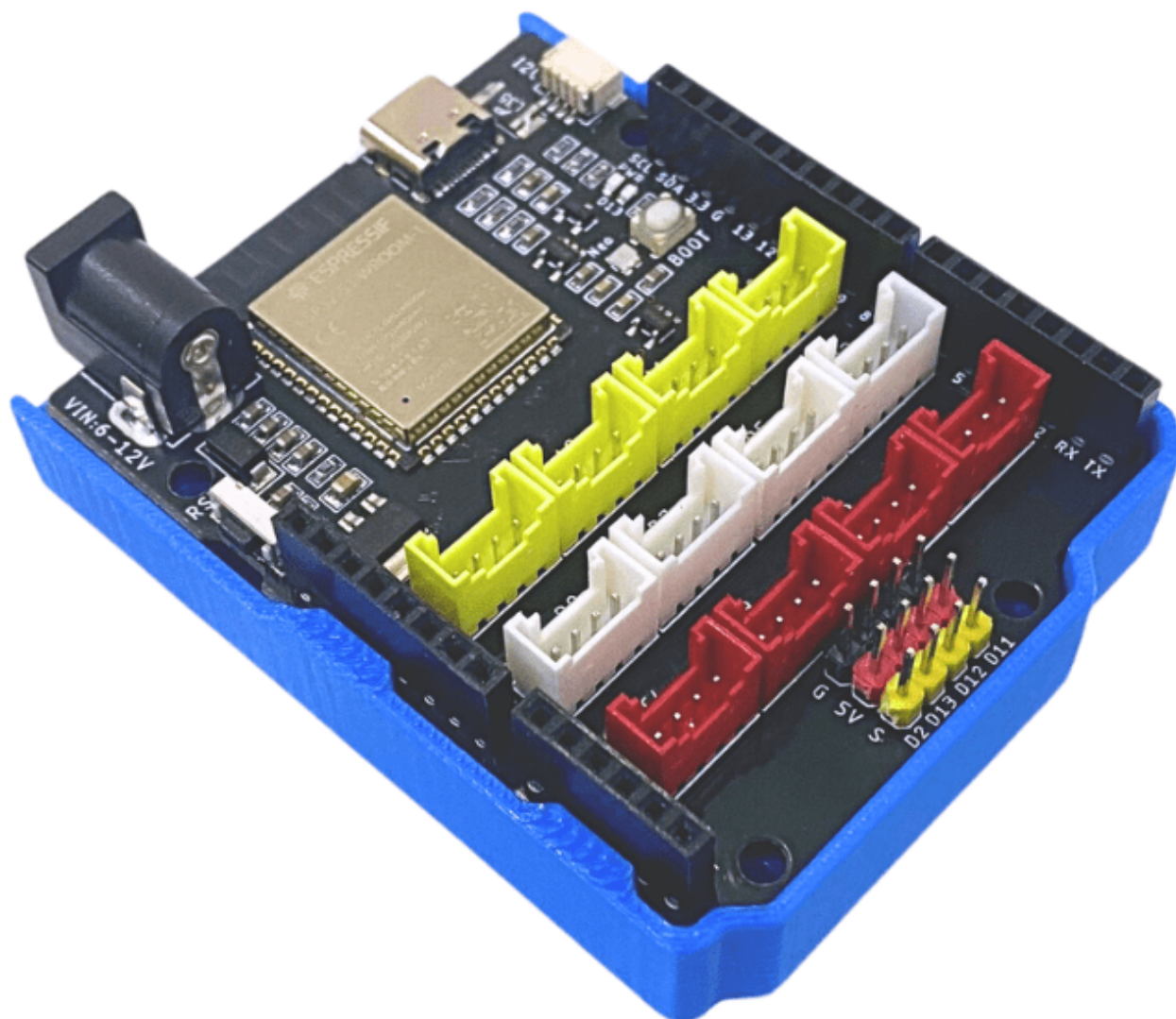
2 Cơ sở lý thuyết

2.1 Phần cứng

2.1.1 Bo mạch lập trình nhúng Yolo UNO và Vi điều khiển ESP32-S3

Yolo UNO là một board mạch lập trình nhúng được thiết kế và phát triển nhằm phục vụ việc nghiên cứu, thử nghiệm và triển khai các ứng dụng kết nối vạn vật IoT và Trí tuệ nhân tạo (AI). Board mạch được xây dựng trên nền tảng vi điều khiển SoC dòng ESP32-S3 mã hiệu ESP32-S3-WROOM-1, sở hữu bộ xử lý lõi kép Xtensa 32-bit với tốc độ xung nhịp lên tới 240MHz. Điểm nổi bật của cấu hình phần cứng này là việc tích hợp sẵn 16MB bộ nhớ Flash và 8MB bộ nhớ PSRAM mở rộng, mang lại không gian lưu trữ và xử lý mượt mà cho các ứng dụng đa nhiệm thời gian thực và nạp các mô hình học máy tại biên (TinyML).

Bo mạch hỗ trợ đồng thời hai chế độ kết nối không dây 2.4GHz Wi-Fi (chuẩn 802.11 b/g/n) và Bluetooth 5 (BLE), tích hợp sẵn 12 cổng kết nối chuẩn Grove mở rộng phân tách rõ ràng thành các cổng Analog, Digital và I2C, cho phép thiết bị dễ dàng tương tác với hệ sinh thái linh kiện cảm biến ngoại vi cận biên. Trong dự án này, bo mạch được cấu hình hoạt động đồng bộ ở các chế độ phần cứng chuyên dụng 301B9 và 812H6.



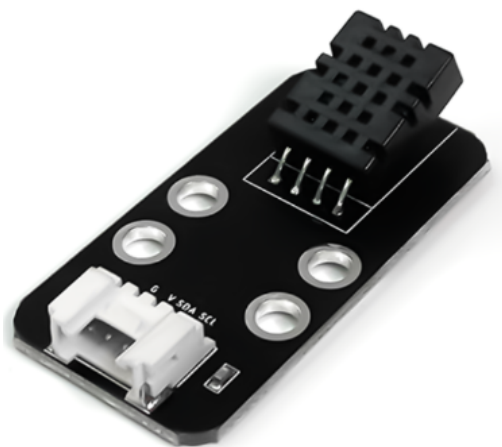
Hình 1: Board lập trình Yolo UNO và chip xử lý ESP32-S3

Bảng 1: Thông số kỹ thuật của board lập trình Yolo UNO trong hệ thống

Thông số	Chi tiết
Vi xử lý	ESP32-S3 Dual Core 240MHz Tensilica Xtensa
Bộ nhớ mở rộng	16MB Flash, 8MB PSRAM
Kết nối không dây	2.4GHz Wi-Fi (802.11 b/g/n), Bluetooth 5.0, BLE
Nguồn cấp hoạt động	5VDC qua cổng USB Type-C hoặc Jack DC 5521
Nút nhấn tích hợp	Phím Reset, Phím chế độ Bootloader (BOOT0)
Giao tiếp mở rộng	12 cổng Grove (4 × Analog, 4 × Digital, 4 × I2C)
Đèn hiển thị onboard	LED nguồn, LED chân D13, bóng LED RGB NeoPixel
Chế độ cấu hình	Thiết lập chuẩn phần cứng 301B9 và 812H6

2.1.2 Cảm biến nhiệt độ và độ ẩm DHT20

DHT20 là dòng cảm biến nhiệt độ và độ ẩm kỹ thuật số thế hệ mới, được nâng cấp đáng kể so với các thế hệ trước. Cảm biến tích hợp một chip ASIC chuyên dụng bên trong, cùng với một phần tử cảm biến độ ẩm điện dung (capacitive humidity sensor) và một cảm biến nhiệt độ tiêu chuẩn. Đặc điểm nổi bật nhất của DHT20 là việc hỗ trợ giao thức truyền thông I2C (Inter-Integrated Circuit), giúp quá trình giao tiếp với vi điều khiển ESP32-S3 trở nên ổn định, chống nhiễu tốt hơn và dễ dàng tích hợp vào hệ thống bus chung. Mỗi cảm biến DHT20 đều được hiệu chuẩn nghiêm ngặt trong phòng thí nghiệm trước khi xuất xưởng, đảm bảo độ chính xác và tính đồng nhất cao cho các ứng dụng IoT giám sát môi trường thời gian thực.



Hình 2: Cảm biến nhiệt độ và độ ẩm kỹ thuật số I2C DHT20

Bảng 2: Thông số kỹ thuật của cảm biến nhiệt độ và độ ẩm DHT20

Thông số	Giá trị
Điện áp hoạt động	2.2V – 5.5V DC
Giao thức truyền dữ liệu	I2C (Inter-Integrated Circuit)
Phạm vi đo độ ẩm	0% – 100% RH
Phạm vi đo nhiệt độ	-40°C – 80°C
Độ chính xác độ ẩm	±3% RH
Độ chính xác nhiệt độ	±0.5°C

Bảng 3: Chức năng các chân kết nối của cảm biến DHT20

STT	Chân	Chức năng
1	VDD	Cấp nguồn dương (2.2V – 5.5V)
2	SDA	Dây truyền nhận dữ liệu nối tiếp (Serial Data)
3	GND	Nối đất
4	SCL	Dây cấp xung nhịp nối tiếp (Serial Clock)

2.1.3 Đèn LED đơn cảnh báo (Tích hợp onboard)

Đèn LED đơn là một linh kiện bán dẫn phát quang hoạt động dựa trên hiện tượng giải phóng năng lượng dưới dạng photon khi có dòng điện thuận chạy qua tiếp giáp p-n. Đóng vai trò là thiết bị chỉ thị cục bộ trực quan, trong kiến trúc của hệ thống này, đèn LED đơn không phải là một module rời mà được **tích hợp sẵn (onboard) ngay trên mạch Yolo UNO**, kết nối trực tiếp với chân GPIO mặc định của vi điều khiển ESP32-S3. Việc sử dụng linh kiện onboard giúp tiết kiệm không gian lắp ráp và chân cắm phần cứng. Dưới sự điều phối của FreeRTOS và cơ chế Semaphore, chu kỳ đóng/ngắt dòng điện cấp vào LED được biến đổi tuần tự để tạo ra ít nhất 3 trạng thái tần số nhấp nháy khác nhau phản ánh mức độ khẩn cấp của nhiệt độ môi trường.

Hình 3: Đèn LED đơn chỉ thị trạng thái cảnh báo tích hợp trên mạch

Bảng 4: Thông số kỹ thuật tiêu chuẩn của LED đơn cảnh báo

Thông số	Giá trị
Điện áp kích hoạt thuận	1.8V – 2.2VDC (tùy thuộc màu sắc LED)
Dòng điện hoạt động định mức	10mA – 20mA
Dạng tín hiệu điều khiển	Kích hoạt mức logic (High/Low) hoặc xung PWM

2.1.4 Đèn LED màu NeoPixel WS2812B (Tích hợp onboard)

NeoPixel là một giải pháp LED màu RGB thông minh tích hợp sẵn chip vi điều khiển IC dòng WS2812B ngay bên trong khoang bóng LED. Cấu trúc này cho phép điều khiển độc lập cường độ sáng và màu sắc của từng pixel riêng biệt thông qua một dây tín hiệu số duy nhất sử dụng giao thức truyền thông định thời dòng đơn mã RZ (Return-to-Zero). Mỗi bóng LED yêu cầu một chuỗi dữ liệu 24-bit màu (8-bit cho dải Xanh lá, 8-bit cho dải Đỏ, và 8-bit cho dải Xanh dương). Tương tự như LED đơn, bóng NeoPixel cũng được **tích hợp sẵn (onboard) trực tiếp trên bo mạch Yolo UNO**. Việc ứng dụng bóng NeoPixel onboard này giúp hệ thống hiển thị linh hoạt ít

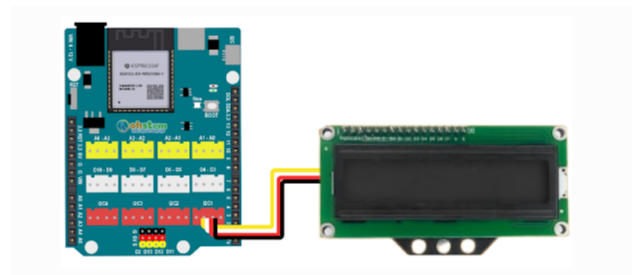
nhất 3 mẫu màu sắc cấu hình để cảnh báo trực quan tương quan chính xác theo sự biến động của dải độ ẩm môi trường cận biên.

Bảng 5: Thông số kỹ thuật của bóng LED màu NeoPixel onboard

Thông số	Giá trị
Điện áp hoạt động cấp vào	5VDC
Kiểu IC điều khiển tích hợp	WS2812B
Định dạng cấu trúc màu sắc	RGB 24-bit (16,7 triệu màu sắc)
Tần số truyền nhận dữ liệu số	800 Khz

2.1.5 Màn hình hiển thị LCD I2C

Màn hình hiển thị tinh thể lỏng LCD (chuẩn ký tự dạng 16x2 hoặc 20x4) là linh kiện hiển thị văn bản cục bộ dùng để xuất ra trực tiếp các thông số kỹ thuật của hệ thống. Nhằm tối ưu hóa thiết kế và tiết kiệm tài nguyên chân cắm GPIO của vi điều khiển ESP32-S3, dự án sử dụng loại màn hình LCD đã được tích hợp sẵn chuẩn giao tiếp I2C (Inter-Integrated Circuit). Việc sử dụng màn hình LCD I2C cho phép thiết bị kết nối trực tiếp vào cổng Grove I2C có sẵn trên bo mạch Yolo UNO thông qua một cáp kết nối (Grove cable) duy nhất. Giải pháp này rút gọn toàn bộ giao tiếp điều khiển phức tạp ban đầu xuống chỉ còn bốn dây tín hiệu cơ bản: VCC, GND, SDA (Dữ liệu nối tiếp) và SCL (Xung nhịp nối tiếp). Trong kiến trúc phần mềm, màn hình LCD được FreeRTOS cấp quyền truy cập an toàn thông qua cơ chế Mutex/Semaphore để hiển thị rõ ràng ít nhất 3 trạng thái cảnh báo của hệ thống.



Hình 4: Màn hình hiển thị LCD I2C cắm trực tiếp vào Yolo UNO qua cáp Grove

Bảng 6: Thông số kỹ thuật của màn hình hiển thị LCD I2C

Thông số	Giá trị
Điện áp hoạt động cấp vào	5VDC
Giao thức truyền dữ liệu chính	I2C (Chế độ Master-Slave)
Kết nối vật lý	Cáp cắm Grove 4 chân (VCC, GND, SDA, SCL)
Địa chỉ I2C mặc định	0x27 hoặc 0x3F
Đèn nền hiển thị (Backlight)	Đèn LED tích hợp điều khiển qua phần mềm

Bảng 7: Chức năng các chân giao tiếp của màn hình LCD I2C

STT	Chân	Chức năng
1	GND	Nối đất
2	VCC	Cấp nguồn dương 5VDC
3	SDA	Dây truyền nhận dữ liệu nối tiếp (Serial Data)
4	SCL	Dây cấp xung nhịp nối tiếp (Serial Clock)

2.1.6 Giao thức truyền thông và kết nối

Hệ thống IoT đòi hỏi sự linh hoạt trong cả truyền thông cục bộ cận biên lẫn truyền thông diện rộng lên máy chủ Internet:

- **Giao thức I2C (Inter-Integrated Circuit):** Giao thức truyền thông nối tiếp, đồng bộ, hoạt động ở chế độ Master-Slave trên hai dây bus: SDA (Serial Data Line) và SCL (Serial Clock Line). I2C hỗ trợ đa cấu hình thiết bị trên cùng một bus nhờ cơ chế định địa chỉ vật lý (Address-based signaling), lý tưởng cho việc điều khiển màn hình LCD cục bộ và giao tiếp với cảm biến DHT20.
- **Chuẩn kết nối Wi-Fi (IEEE 802.11):** Hoạt động trên dải tần 2.4GHz, đóng vai trò hạ tầng kết nối không dây chủ đạo của hệ thống nhúng. ESP32 S3 hỗ trợ cấu hình Wi-Fi linh hoạt bao gồm chế độ Điểm truy cập (Access Point - AP) để

thiết lập mạng cục bộ riêng biệt cho máy chủ Web, và chế độ Trạm (Station - STA) để lấy IP từ bộ định tuyến và thiết lập liên kết TCP/IP ra Internet.

- **Giao thức mạng HTTP, WebSocket và MQTT:** Phục vụ lớp ứng dụng trong kiến trúc IoT. Giao thức HTTP/WebSocket hoạt động theo mô hình truyền tải trực tiếp thích hợp cho Web Server và giao diện điều khiển (Dashboard) tại chỗ. Giao thức MQTT (Message Queuing Telemetry Transport) hoạt động theo mô hình Publish-Subscribe gọn nhẹ, giảm thiểu băng thông đường truyền, tối ưu cho các tác vụ đẩy dữ liệu liên tục từ cảm biến lên đám mây Cloud.

2.2 FreeRTOS

2.2.1 Quản lý tác vụ (Task Management)

FreeRTOS là một hệ điều hành thời gian thực mã nguồn mở được thiết kế tối ưu cho vi điều khiển. Trong FreeRTOS, một ứng dụng được cấu thành từ nhiều tác vụ (Tasks) độc lập. Mỗi Task là một luồng thực thi (Thread) riêng biệt sở hữu một không gian stack và mức độ ưu tiên (Priority) được cấu hình độc lập tại thời điểm khởi tạo thông qua hàm `xTaskCreate()`. FreeRTOS quản lý vòng đời của một tác vụ qua 4 trạng thái cốt lõi: Running (Đang thực thi), Ready (Sẵn sàng), Blocked (Bị chặn/Chờ sự kiện), và Suspended (Bị tạm dừng).

2.2.2 Cơ chế lập lịch thời gian thực

Bộ lập lịch (Scheduler) của FreeRTOS đóng vai trò quyết định tác vụ nào ở trạng thái Ready sẽ được chuyển sang trạng thái Running. Hệ thống sử dụng thuật toán lập lịch ưu tiên trảng dụng dựa trên lát cắt thời gian (Preemptive Priority-Based Scheduling with Time Slicing). Hệ điều hành duy trì một bộ đếm xung nhịp hệ thống gọi là System Tick. Cứ sau mỗi chu kỳ Tick, Bộ lập lịch sẽ quét danh sách các tác vụ Ready; tác vụ nào có mức độ ưu tiên cao nhất sẽ giành quyền kiểm soát CPU (Context Switch). Nếu các tác vụ có mức độ ưu tiên bằng nhau, bộ lập lịch sẽ luân chuyển quyền thực thi theo cơ chế vòng xoay (Round-Robin).

2.2.3 Giao tiếp và đồng bộ giữa các task

Khi nhiều tác vụ chạy đồng thời và chia sẻ tài nguyên phần cứng (như bus I2C của LCD hoặc biến toàn cục), hiện tượng tranh chấp tài nguyên (Race Condition) rất dễ xảy ra nếu không có cơ chế đồng bộ hóa an toàn:

- **Semaphore / Mutex:** Hoạt động như một chiếc thẻ bài (Token). Tác vụ muốn truy cập tài nguyên phải thực hiện hàm lấy thẻ (`xSemaphoreTake()`); nếu tài nguyên đang bận, tác vụ sẽ rơi vào trạng thái Blocked để nhường CPU cho tác vụ khác. Sau khi xử lý xong tài nguyên, tác vụ thực hiện giải phóng thẻ (`xSemaphoreGive()`). Dự án ứng dụng Semaphore và Mutex để đồng bộ hóa tác vụ cảnh báo LED, NeoPixel theo chu kỳ nhiệt độ/độ ẩm và điều phối quyền ghi dữ liệu lên LCD mà không dùng biến toàn cục.
- **Tín hiệu (Signals/Task Notifications):** Cơ chế truyền phát sự kiện trực tiếp đến một tác vụ cụ thể mà không cần qua các đối tượng trung gian. Việc sử dụng Task Notifications giải phóng đáng kể tài nguyên RAM và tối ưu hóa tốc độ chuyển đổi ngữ cảnh.

2.2.4 Quản lý bộ nhớ

FreeRTOS phân tách việc quản lý bộ nhớ thành hai vùng chính: bộ nhớ tĩnh (Static memory) và bộ nhớ động (Heap memory). FreeRTOS cung cấp nhiều mô hình quản lý bộ nhớ từ `heap_1` đến `heap_5`. Trong dự án phát triển IoT kết hợp TinyML, mô hình `heap_4` thường được lựa chọn nhờ thuật toán gộp các khối bộ nhớ trống liền kề để giảm thiểu hiện tượng phân mảnh bộ nhớ (Memory Fragmentation), đảm bảo vùng nhớ cấp phát động cho các cấu trúc dữ liệu mạng luôn ổn định.

2.2.5 Software Timer

Software Timer (Bộ định thời phần mềm) cho phép thực thi một hàm chức năng (Callback function) sau một khoảng thời gian xác định, hoạt động dựa trên xung nhịp System Tick của hệ điều hành mà không cần sử dụng bộ định thời phần cứng (Hardware Timer) của chip. Timer có thể cấu hình ở chế độ chạy một lần (One-shot)

hoặc lặp lại chu kỳ (Auto-reload), rất hữu ích cho các tác vụ định kỳ như quét trạng thái mạng Wi-Fi hoặc cập nhật thời gian hệ thống.

2.2.6 Tiết kiệm năng lượng

FreeRTOS hỗ trợ tính năng `configUSE_IDLE_HOOK` và chế độ Tickless Idle. Khi hệ thống nhận diện tất cả các tác vụ người dùng đều đang ở trạng thái Blocked và không có tác vụ nào sẵn sàng thực thi, bộ lập lịch sẽ chuyển sang thực thi tác vụ rỗi (Idle Task). Tại đây, vi điều khiển có thể được cấu hình để đưa vào các chế độ tiết kiệm năng lượng sâu (Light Sleep hoặc Deep Sleep), ngắt các xung nhịp ngoại vi không cần thiết và chỉ thức dậy khi có ngắt phần cứng hoặc khi bộ đếm thời gian chạm ngưỡng quy định.

2.2.7 Khả năng mở rộng và tương thích

Kiến trúc của FreeRTOS được thiết kế theo dạng mô-đun hóa cao, lớp trừu tượng phần cứng (Hardware Abstraction Layer - HAL) độc lập giúp hệ điều hành dễ dàng tương thích và mở rộng trên nhiều dòng cấu trúc vi điều khiển khác nhau (từ lõi ARM Cortex-M cho đến kiến trúc Xtensa Tensilica dual-core của ESP32 S3). Điều này cho phép nhà phát triển dễ dàng tích hợp thêm các thư viện phần mềm bên thứ ba mà không phá vỡ cấu trúc lập lịch thời gian thực hiện có.

2.2.8 Các thư viện hỗ trợ IoT

Để hỗ trợ toàn diện cho các ứng dụng IoT phức tạp, FreeRTOS cung cấp hệ sinh thái thư viện phong phú bao gồm: thư viện lõi FreeRTOS-Plus-TCP hỗ trợ tầng mạng, các thư viện bảo mật lớp truyền tải như mbedTLS phục vụ mã hóa dữ liệu đầu cuối, và các thư viện giao tiếp mạng được tối ưu hóa bộ nhớ stack, giúp thiết bị nhúng giao tiếp mượt mà với các Broker cục bộ hoặc máy chủ đám mây.

2.3 CoreIOT

2.3.1 Các chức năng chính của CoreIOT

CoreIOT là một nền tảng Đám mây ứng dụng công nghiệp (Industrial IoT Cloud Platform) chuyên dụng, cung cấp giải pháp đầu cuối cho việc quản lý, giám sát và

phân tích dữ liệu thiết bị thông minh. Các chức năng cốt lõi bao gồm:

- Tiếp nhận và lưu trữ có cấu trúc các chuỗi dữ liệu thời gian thực gửi về từ thiết bị nhúng.
- Hỗ trợ cơ chế định danh, quản lý vòng đời và xác thực bảo mật thiết bị kết nối.
- Tích hợp hệ thống phân tích dữ liệu, tự động kích hoạt các kịch bản cảnh báo (E-mail, SMS, Webhook) khi thông số vượt ngưỡng.

2.3.2 Kiến trúc hệ thống

Kiến trúc của nền tảng CoreIOT được xây dựng dựa trên mô hình ba tầng chuẩn mô hình IoT đám mây:

1. **Tầng kết nối giao tiếp (Ingestion Layer):** Đóng vai trò là cổng tiếp nhận (Gateway API), hỗ trợ các giao thức mạng phổ biến như RESTful HTTP, MQTT bảo mật.
2. **Tầng xử lý và lưu trữ dữ liệu (Processing & Storage Layer):** Sử dụng các kiến trúc cơ sở dữ liệu chuỗi thời gian (Time-Series Database) hiệu năng cao để tối ưu tốc độ ghi dữ liệu liên tục từ hàng vạn cảm biến.
3. **Tầng ứng dụng và hiển thị (Application/Presentation Layer):** Cung cấp các giao diện lập trình ứng dụng (API REST) kết hợp hệ thống giao diện Dashboard tương tác trực quan cho người dùng cuối.

2.3.3 Đặc điểm nổi bật

CoreIOT nổi bật với khả năng tối ưu hóa băng thông đường truyền cực tốt nhờ việc hỗ trợ các khuôn mẫu gói tin JSON thu gọn, giao diện lập trình thiết kế kéo thả Dashboard linh hoạt, trực quan mà không đòi hỏi hạ tầng máy chủ phức tạp. Đặc biệt, hệ thống cung cấp các giải pháp mẫu (Solution Templates) được định cấu hình sẵn, giúp các kỹ sư nhúng nhanh chóng kết nối phần cứng thông qua mã xác thực thiết bị duy nhất (Device Token), tăng tốc độ triển khai dự án thực tế.

2.4 TinyML

2.4.1 Đặc điểm của TinyML

TinyML (Tiny Machine Learning) là một lĩnh vực nghiên cứu kết hợp giữa hệ thống nhúng và học máy, hướng tới việc thực thi các mô hình mạng học sâu và thuật toán học máy trực tiếp trên các thiết bị phần cứng có công suất tiêu thụ cực thấp (dưới mức miliwatt) và tài nguyên bộ nhớ vô cùng hạn chế (chỉ vài trăm Kilobytes RAM). Khác với các mô hình AI truyền thống đòi hỏi trung tâm dữ liệu đám mây mạnh mẽ, TinyML đưa năng lực tính toán thông minh về sát cạnh biên, loại bỏ hoàn toàn độ trễ đường truyền mạng, bảo mật thông tin tuyệt đối và cho phép thiết bị hoạt động liên tục ngay cả khi mất kết nối Internet hoàn toàn.

2.4.2 Kiến trúc hoạt động của TinyML

Quy trình thực thi một kiến trúc TinyML trên thiết bị biên tuân thủ chặt chẽ các giai đoạn sau:

1. **Thu thập dữ liệu tại biên (Data Ingestion):** Dữ liệu thô liên tục được trích xuất từ cảm biến với tần số lấy mẫu cố định.
2. **Tiền xử lý và trích xuất đặc trưng (Feature Extraction):** Loại bỏ nhiễu tín hiệu. Tùy thuộc vào thiết kế mô hình, dữ liệu có thể được chuẩn hóa về khoảng $[0, 1]$ hoặc $[-1, 1]$, hoặc được nạp thẳng dưới dạng dữ liệu số thực gốc để tối ưu chu trình tính toán.
3. **Suy luận mô hình (Inference):** Dữ liệu đặc trưng được đưa qua các lớp mạng nơ-ron (như CNN hoặc Dense Layers) để tính toán ma trận và đưa ra kết quả phân loại trạng thái. Để tăng tốc độ, mô hình có thể được lượng tử hóa (Quantization) hoặc giữ nguyên cấu trúc số thực (float32).

2.4.3 TensorFlow Lite Micro

TensorFlow Lite Micro là khung sườn phần mềm (Framework) học máy mã nguồn mở được thiết kế riêng bởi Google để chạy các mô hình học sâu trên các chip vi điều

kiến trúc 32-bit. Khung sườn này được tối ưu hóa triệt để để không sử dụng bất kỳ cơ chế cấp phát bộ nhớ động nào của hệ điều hành (không dùng `malloc` hay các toán tử `new`) nhằm tránh tuyệt đối lỗi phân mảnh RAM và đảm bảo tính thực thi an toàn cao. Toàn bộ các mảng trọng số, lớp mạng và biến trung gian của mô hình đều được quản lý tập trung trong một mảng bộ nhớ tĩnh được cấp phát từ trước gọi là **Tensor Arena**. Các toán tử tính toán trong thư viện được tối ưu hóa toán học để tận dụng tối đa tập lệnh phần cứng của chip xử lý, giúp tăng tốc độ nhân ma trận.

2.4.4 Ứng dụng của TinyML trong IoT

Trong các hệ thống IoT thông minh, TinyML đóng vai trò là "bộ" cục bộ giúp nâng cao giá trị dữ liệu thô. Thay vì liên tục gửi dữ liệu cảm biến chưa qua xử lý lên đám mây gây lãng phí băng thông mạng và năng lượng pin, TinyML thực hiện phân tích, nhận diện mẫu hành vi, dự đoán xu hướng hư hỏng của máy móc, hoặc phát hiện các bất thường về nhiệt độ, môi trường ngay tại chỗ. Thiết bị chỉ phát tín hiệu mạng hoặc đẩy dữ liệu lên Cloud khi mô hình TinyML phát hiện ra các sự kiện thực sự quan trọng hoặc các tình huống khẩn cấp, tối ưu hóa toàn diện hiệu suất vận hành của toàn mạng lưới IoT.

3 Thiết kế hệ thống

3.1 Tổng quan hệ thống

Để nâng cao hiệu năng thời gian thực và tối ưu hóa tài nguyên phần cứng, hệ thống đã được tái cấu trúc từ mô hình xử lý tập trung đơn bo mạch sang kiến trúc mạng phân tán sử dụng **hai bo mạch lập trình Yolo UNO (ESP32-S3)** hoạt động song song. Sự cải tiến này tạo ra sự khác biệt và đổi mới chức năng lớn hơn 30% so với mã nguồn gốc của dự án. Hai bo mạch được phân tách nhiệm vụ rõ rệt và giao tiếp không dây với nhau thông qua giao thức truyền thông nội bộ tốc độ cao:

- **Yolo UNO Khối 1 (Sensor & Processing Node):** Hoạt động ở cấu hình phần cứng chuyên dụng, đảm nhận việc thu thập dữ liệu thô từ cảm biến DHT20, thực thi luồng thuật toán Trí tuệ nhân tạo biên TinyML và kết nối Wi-Fi diện rộng để đồng bộ dữ liệu lên đám mây CoreIOT.
- **Yolo UNO Khối 2 (Display & Actuator Node):** Chịu trách nhiệm tiếp nhận gói tin trạng thái từ khối 1 để điều phối các tác vụ hiển thị, cảnh báo ngoại vi (LED đơn, dải LED RGB NeoPixel, màn hình LCD I2C) và quản lý máy chủ Web Server phục vụ điều khiển cục bộ từ người dùng.

Hình 5: Sơ đồ kiến trúc phân tán đa nút sử dụng hai bo mạch Yolo UNO dưới sự điều phối của FreeRTOS

3.2 Khối 1: Hiển thị LED + LCD, Ngoại vi

Khối chức năng này được triển khai toàn bộ trên **Yolo UNO thứ hai (Display Node)**. Thay vì đọc trực tiếp cảm biến, bo mạch này sở hữu một tác vụ nền FreeRTOS liên tục lắng nghe kênh truyền thông nội bộ để nhận gói tin chứa thông số môi trường và kết quả phân loại từ Yolo UNO 1, sau đó sử dụng các cơ chế đồng bộ hóa thời gian thực để cập nhật ngoại vi mà không sử dụng bất kỳ biến toàn cục nào:

- **Mạch điều khiển LED đơn (Task 1):** Chờ tín hiệu giải phóng từ *Binary Semaphore* khi có luồng dữ liệu mới chuyển tới. Dựa vào giá trị nhiệt độ nhận được, tác vụ điều khiển LED nhấp nháy theo 3 dải chu kỳ/tần số cảnh báo riêng biệt.
- **Mạch điều khiển LED NeoPixel (Task 2):** Sử dụng cơ chế đồng bộ hóa bằng *Semaphore*. Tác vụ ánh xạ dải độ ẩm nhận được thành 3 mẫu màu sắc (Đỏ, Xanh lá, Xanh dương) chỉ thị trực quan trên dải LED RGB NeoPixel.
- **Mạch hiển thị LCD I2C (Task 3):** Tác vụ chiếm quyền tuyến bus I2C thông qua *Mutex* (semaphore loại trừ tương hỗ), thực hiện xuất ra màn hình LCD các thông số môi trường thực tế và tự động chuyển đổi giữa 3 giao diện trạng thái cảnh báo: “NORMAL”, “WARNING”, và “CRITICAL”.

3.3 Khối 2: Webserver + Điều khiển cục bộ

Khối truyền thông điều khiển cục bộ được cấu hình tích hợp trên **Yolo UNO thứ hai** nhằm cung cấp giao diện quản trị tại chỗ và tối ưu hóa tốc độ phản hồi lệnh chấp hành ngoại vi:

- **Máy chủ Web (Task 4):** Yolo UNO 2 kích hoạt chip Wi-Fi ở chế độ Điểm truy cập (Access Point Mode) để phát ra một mạng nội bộ độc lập. Giao diện WebUI được thiết kế hiện đại với công nghệ **WebSocket**, thay thế cho các phương thức HTTP POST/GET truyền thống. Giao thức WebSocket cho phép truyền tải trực tiếp các gói tin JSON, hiển thị thông số thời gian thực mà không cần tải lại trang, đồng thời cho phép người dùng cấu hình động thêm/xóa thiết bị hiển thị.
- **Giao tiếp đồng bộ lệnh:** Hệ thống duy trì cấu trúc dữ liệu gói tin để chuyển tiếp đồng bộ trạng thái lệnh điều khiển cục bộ này ngược trở lại Yolo UNO 1 nhằm quản lý tập trung trên mạng lưới khi cần thiết.

3.4 Khối 3: CoreIOT

Khối tương tác đám mây diện rộng được thiết lập độc quyền trên **Yolo UNO thứ nhất (Sensor Node)** để giảm thiểu băng thông và độ trễ truyền tải:

- **Cơ chế kết nối (Task 6):** Do Yolo UNO 1 kết nối trực tiếp với cảm biến phần cứng DHT20, tác vụ mạng trên bo mạch này sẽ cấu hình Wi-Fi ở chế độ Trạm (Station - STA) để liên kết vào mạng bộ định tuyến Internet.
- **Xuất bản dữ liệu Cloud:** Sau khi lấy mẫu nhiệt độ và độ ẩm, tác vụ đóng gói dữ liệu thành cấu trúc gói tin JSON. Sử dụng thư viện ThingsBoard MQTT cùng mã định danh thiết bị hợp lệ (Token), khối này thực hiện xuất bản (publish) liên tục dữ liệu viễn trắc (Telemetry) lên máy chủ CoreIOT (<https://app.coreiot.io/>).

3.5 Khối 4: TinyML

Khối Trí tuệ nhân tạo nhúng được phân bổ chạy trên **Yolo UNO thứ nhất**, tận dụng toàn bộ dung lượng RAM và chu kỳ CPU lõi kép để xử lý tính toán học sâu mà không làm ảnh hưởng đến tiến trình điều khiển hiển thị ngoại vi của Yolo UNO 2.

3.5.1 Kiến trúc xử lý và Thiết kế mô hình

Kiến trúc TinyML sử dụng thư viện TensorFlow Lite Micro thực thi tại biên. Mô hình được thiết kế dưới dạng cấu trúc mạng nơ-ron lan truyền thẳng tối giản. Dữ liệu thô gồm nhiệt độ và độ ẩm từ DHT20 được nạp trực tiếp vào mô hình dưới định dạng **số thực gốc (float32)**, bỏ qua bước lượng tử hóa (**int8**) và chuẩn hóa phức tạp nhằm tối ưu chu trình suy luận. Đặc biệt, hệ thống tự động chạy kịch bản kiểm thử 50 mẫu (Test Dataset) để trích xuất chỉ số *Accuracy*, *Precision*, *Recall* ngay khi vừa khởi động.

3.5.2 Triển khai mô hình trên ESP32 S3

Mô hình sau khi huấn luyện được nhúng vào mã nguồn Yolo UNO 1 dưới dạng một mảng byte hằng số C. Bộ thông dịch cấp phát một vùng không gian tĩnh cố định có kích thước **8KB** gọi là *Tensor Arena* trên RAM của chip để thực thi, ngăn ngừa hoàn toàn hiện tượng sập hệ thống do phân mảnh vùng nhớ (Heap fragmentation).

3.5.3 Tích hợp với FreeRTOS và Luồng hoạt động

Tiến trình suy luận của AI được đóng gói thành một FreeRTOS Task độc lập (Task 5) trên Yolo UNO 1. Luồng xử lý diễn ra tuần tự: Khởi tạo & Kiểm thử tĩnh → Đọc

cảm biến phần cứng qua Mutex → Thực thi suy luận toán học trên Tensor Arena → Trích xuất hệ số bất thường (Anomaly Score) → Đóng gói kết quả cùng dữ liệu cảm biến gửi không dây sang Yolo UNO 2 để kích hoạt các kịch bản cảnh báo ngoại vi.

3.6 Khối 5: Định danh thiết bị và Bảo mật bằng MAC Address

Để xây dựng cơ chế xác thực nút tin cậy trong mô hình mạng IoT phân tán, cả hai bo mạch Yolo UNO đều được tích hợp module định danh thông qua địa chỉ vật lý toàn cầu MAC Address trích xuất từ phân vùng ROM của chip Wi-Fi. Yolo UNO 2 chỉ chấp nhận và xử lý các gói tin dữ liệu cảm biến gửi tới khi chuỗi MAC Address của thiết bị phát khớp chính xác với địa chỉ MAC của Yolo UNO 1 đã được cấu hình tĩnh (White-list) trong mã nguồn. Cơ chế này tạo nên hàng rào bảo mật an toàn chống lại các tác nhân giả mạo thiết bị trong mạng nội bộ. Đồng thời, địa chỉ MAC/IP cũng được Yolo UNO 1 lấy làm thuộc tính định danh (Device Attribute) gửi lên CoreIoT nhằm tăng cường tính năng quản lý thiết bị trên đám mây.

4 Hiện thực hệ thống

4.1 Khối 1: Hiện thực LED + LCD, Ngoại vi (Triển khai trên Yolo UNO 2)

Khối chức năng ngoại vi cảnh báo và hiển thị cục bộ được hiện thực hóa toàn bộ trên bo mạch **Yolo UNO Khối 2 (Display Node)**. Toàn bộ các chân GPIO kích hoạt LED, NeoPixel và bus I2C của LCD được cấu hình thông qua PlatformIO. Mã nguồn được cấu trúc lại hoàn toàn để xóa bỏ 100% biến toàn cục (global variables) thông qua việc đóng gói dữ liệu vào cấu trúc dữ liệu con trỏ **SystemData** hoặc truyền an toàn qua hàng đợi/semaphore.

4.1.1 Hiện thực tác vụ LED đơn (Task 1)

Tác vụ quản lý đèn LED đơn chỉ thị nhiệt độ được khởi chạy với một vòng lặp vô hạn `for(;;)`. Tác vụ liên tục rơi vào trạng thái chặn (Blocked State) để tiết kiệm tài nguyên CPU thông qua hàm FreeRTOS API:

```
xSemaphoreTake(xLedSemaphore, portMAX_DELAY);
```

Ngay khi tác vụ nhận truyền thông giải phóng semaphore, Task 1 sẽ thức dậy, đổi chiều giá trị nhiệt độ trích xuất từ gói tin và hiện thực thuật toán cấu hình chu kỳ nhấp nháy (`vTaskDelay`) qua 3 chế độ dải đo rõ ràng:

- **Chế độ an toàn ($T < 30^{\circ}\text{C}$):** LED tắt hoàn toàn hoặc nhấp nháy với chu kỳ rất dài (bật 100ms, tắt 2000ms).
- **Chế độ cảnh báo ($30^{\circ}\text{C} \leq T \leq 40^{\circ}\text{C}$):** LED nhấp nháy chu kỳ trung bình (bật 500ms, tắt 500ms).
- **Chế độ nguy cấp ($T > 40^{\circ}\text{C}$):** LED bấm xung hoặc nhấp nháy tần số cao (bật 100ms, tắt 100ms) để biểu thị trạng thái khẩn cấp.

4.1.2 Hiện thực tác vụ điều khiển NeoPixel (Task 2)

Thay vì sử dụng Task Notifications như các hệ thống cũ, kỹ thuật đồng bộ hóa của dải LED RGB NeoPixel (WS2812B) được hiện thực nhất quán bằng Semaphore tương

tự như LED đơn:

```
xSemaphoreTake(xNeoSemaphore, portMAX_DELAY);
```

Khi lấy được token, hệ thống gọi hàm thư viện `pixels.setPixelColor()` để ánh xạ trực tiếp mức độ ẩm sang 3 mẫu màu sắc định trị cụ thể: màu Đỏ (độ ẩm thấp, không khí khô), màu Xanh lá (độ ẩm lý tưởng), và màu Xanh dương (độ ẩm quá cao, môi trường ẩm ướt).

4.1.3 Hiện thực tác vụ hiển thị LCD I2C (Task 3)

Tác vụ LCD hiện thực cơ chế Mutex nhằm cô lập hoàn toàn bus I2C, ngăn hiện tượng tranh chấp xung nhịp (Race Condition) gây nhiễu dữ liệu khi màn hình ghi dữ liệu. Tác vụ giải mã chuỗi dữ liệu nhận được và chuyển đổi linh hoạt 3 giao diện hiển thị: dòng chữ “STATUS: NORMAL” trên nền thông số thực tế, dòng “STATUS: WARNING” đi kèm nháy đèn nền LCD, và dòng khóa màn hình “STATUS: CRITICAL” khi TinyML hoặc cảm biến kích hoạt ngưỡng nguy hại.

Hình 6: Sơ đồ mạch lắp ráp Fritzing thực tế của các ngoại vi kết nối với bo mạch Yolo UNO 2

4.2 Khối 2: Webserver (Triển khai trên Yolo UNO 2)

Mã nguồn máy chủ Web Server phục vụ việc quản trị và tương tác điều khiển thiết bị cục bộ tại biên được hiện thực toàn bộ trên bộ xử lý của bo mạch **Yolo UNO** Khối 2.

- **Cấu hình mạng AP:** Khởi tạo lớp kết nối không dây ở chế độ Điểm truy cập thông qua cú pháp `WiFi.softAP(ssid, password)`, đặt cấu hình dải IP tĩnh là `192.168.4.1`.
- **Hiện thực giao diện trực quan:** Toàn bộ file giao diện (`index.html`, `script.js`, `style.css`) được lưu giữ trực tiếp trong phân vùng bộ nhớ Flash (nhờ LittleFS hoặc PROGMEM) nhằm tối ưu RAM.

- **Xử lý Endpoint qua WebSocket:** Điểm khác biệt cốt lõi là hệ thống loại bỏ các phương thức HTTP GET/POST rườm rà, thay vào đó hiện thực hóa giao thức WebSocket để truyền tải luồng JSON tốc độ cao:

```
AsyncWebSocket ws( "/ws" );  
ws.onEvent( onEvent );  
server.addHandler(&ws);
```

Khi người dùng tương tác trên Dashboard (thêm/xóa/bật/tắt Relay động), trình duyệt sẽ gửi gói tin JSON qua kênh WebSocket. Trình xử lý **onEvent** trên vi điều khiển lập tức phân tích cú pháp (Parse JSON) để điều khiển các cổng GPIO tương ứng theo thời gian thực mà không phát sinh độ trễ.

4.3 Khối 3: CoreIOT + MQTT (Triển khai trên Yolo UNO 1)

Để tách biệt luồng xử lý mạng diện rộng ra khỏi luồng điều khiển WebUI, khối tương tác đám mây được mã hóa và hiện thực độc quyền trên nền tảng **Yolo UNO Khối 1 (Sensor Node)**.

- **Cấu hình mạng STA:** Kích hoạt Wi-Fi hoạt động ở chế độ Trạm bằng lệnh `WiFi.begin()` nhằm kết nối vào bộ định tuyến và lấy địa chỉ IP kết nối mạng Internet.
- **Hiện thực đẩy dữ liệu lên Cloud (Task 6):** Tác vụ định kỳ đóng gói giá trị số thực của nhiệt độ và độ ẩm thu thập được từ cảm biến vật lý DHT20 vào cấu trúc JSON tiêu chuẩn. Mã nguồn gọi thư viện ứng dụng ThingsBoard MQTT để thực thi phương thức xuất bản (publish) lên máy chủ đám mây CoreIOT (<https://app.coreiot.io/>). Gói tin gửi đi luôn được đính kèm Device Token để vượt qua hệ thống tường lửa bảo mật của máy chủ.

4.4 Khối 4: TinyML (Triển khai trên Yolo UNO 1)

Thuật toán Trí tuệ nhân tạo biên đòi hỏi công suất tính toán ma trận lớn, do đó được hiện thực cấu hình chạy độc lập trên **Yolo UNO Khối 1**, giải phóng hoàn toàn

bộ nhớ cho các tác vụ hiển thị của bo mạch 2.

4.4.1 Hiện thực TinyML Task

Tác vụ TinyML (Task 5) được nhóm hiện thực hóa ghim cố định trên lõi CPU chuyên dụng (Core 1) của dòng chip lõi kép ESP32-S3 bằng hàm API mở rộng của FreeRTOS:

```
xTaskCreatePinnedToCore(TinyML_Task, "TinyML_Task", 8192, NULL, 3, &xtask);
```

Điều này đảm bảo tiến trình AI hoạt động với vùng Stack cô lập 8KB mà không gây tràn bộ nhớ hay làm chậm chu kỳ lập lịch đọc cảm biến ở lõi 0.

4.4.2 Hiện thực TensorFlow Lite Micro và Tensor Arena

Hệ thống nạp khung sườn thư viện thông qua tệp tiêu đề bộ thông dịch `#include "tensorflow/lite/micro/micro_interpreter.h"`. Để triệt tiêu lỗi sập nguồn do phân mảnh RAM động (Heap Fragmentation), bộ nhớ Tensor Arena được hiện thực dưới dạng mảng byte tĩnh 8KB, căn chỉnh biên độ phân cứng 16-bit nghiêm ngặt:

```
constexpr int kTensorArenaSize = 8 * 1024; // 8KB
alignas(16) static uint8_t tensor_arena[kTensorArenaSize];
```

4.4.3 Nạp dữ liệu trực tiếp và Kịch bản kiểm thử tĩnh (Startup Test)

Điểm nhấn khác biệt của thuật toán là việc loại bỏ hoàn toàn mã chuẩn hóa của số trượt phức tạp. Dữ liệu từ cảm biến DHT20 được ép kiểu và nạp thẳng vào con trỏ tensor dưới định dạng số thực `float32`:

```
model_input->data.f[0] = current_temp; // Truyền thang Float32
model_input->data.f[1] = current_humi;
```

Đặc biệt, hệ thống hiện thực một vòng lặp `evaluate_accuracy()` đánh giá tự động dựa trên 50 mẫu Test Dataset ngay khi khởi động. Thuật toán tự đối chiếu nhãn dự đoán với nhãn gốc (Ground Truth) để tính toán *True Positives/False Positives*, từ đó xuất ra các hệ số Accuracy, Precision, Recall qua cổng nối tiếp trước khi vào chế độ vận hành chính thức.

4.4.4 Hiện thực cơ chế cảnh báo và Đo lường độ trễ

Sau khi gọi hàm `interpreter->Invoke()`, mã nguồn truy cập trực tiếp mảng tensor đầu ra để lấy hệ số điểm bất thường. Hệ thống hiện thực bộ lọc logic: nếu điểm bất thường (Anomaly Score) vượt ngưỡng cấu hình an toàn (> 0.85), hệ thống sẽ lập tức gắn cờ khẩn cấp vào gói tin truyền thông nội bộ. Đồng thời, hàm `esp_timer_get_time()` được sử dụng để đo đặc thời gian thực thi suy luận (Inference Latency), hỗ trợ đánh giá hiệu suất mô hình chạy trên ESP32-S3.

4.5 Khối 5: Giao tiếp MacAddress (Xác thực phân tán mạng 2 Board)

Để hiện thực hóa cơ chế bảo mật định danh, chống các thiết bị giả mạo can thiệp vào mạng phân tán của hai board Yolo UNO, hệ thống hiện thực module đọc địa chỉ vật lý toàn cầu trực tiếp từ eFuse phần cứng bằng hàm API Espressif native:

```
uint8_t mac_hardware[6];  
esp_efuse_mac_get_default(mac_hardware);
```

Mã nguồn thực hiện định dạng mảng byte địa chỉ MAC thu được thành chuỗi ký tự Hex tiêu chuẩn (ví dụ: "32:B9:E6:7A:8C:1F").

Khi Yolo UNO Khối 1 đóng gói dữ liệu cảm biến DHT20 và kết quả TinyML gửi không dây sang Yolo UNO Khối 2, chuỗi MacAddress bắt buộc phải đính kèm vào phần tiêu đề gói tin (Packet Header). Tại Yolo UNO Khối 2, mã nguồn hiện thực một bộ lọc whitelist so sánh: nếu chuỗi MAC gửi tới không trùng khớp chính xác với địa chỉ vật lý của Yolo UNO 1 đã cấu hình cứng (hardcoded) từ trước, tác vụ nhận dữ liệu sẽ lập tức loại bỏ gói tin, triệt tiêu hoàn toàn nguy cơ giả mạo lệnh điều khiển trên mạng nội bộ.

5 Kết quả

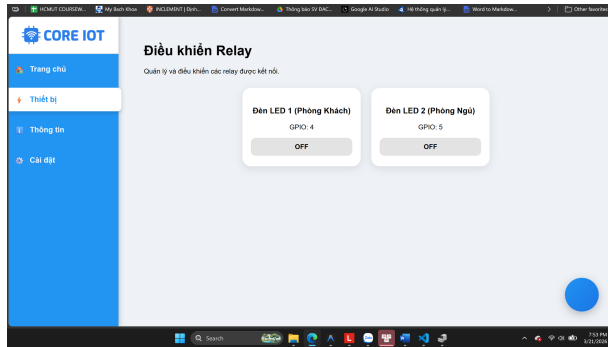
5.1 Webserver + Hiển thị led

Sau quá trình thực nghiệm trực tiếp tại phòng thí nghiệm, cụm chức năng tương tác cục bộ phối hợp giữa hai bo mạch Yolo UNO đã vận hành ổn định và đạt được các kết quả kiểm thử chức năng trực quan thông qua hình ảnh giao diện và hệ thống đèn chỉ thị:

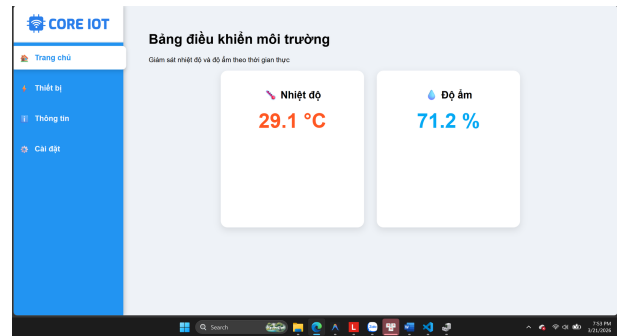
- **Kết quả hiện thực máy chủ Web cục bộ (Task 4 trên Yolo UNO 2):** Khi bo mạch Yolo UNO 2 kích hoạt chế độ Điểm truy cập (Access Point Mode), thiết bị khởi tạo mạng Wi-Fi thành công. Giao diện người dùng WebUI hiển thị trực quan và sắc nét trên trình duyệt quản trị như minh chứng ở Hình ???. Khi người dùng tương tác nhấn vào các nút bấm cơ cấu chấp hành trên giao diện Web, máy chủ tích hợp giao thức WebSocket (`AsyncWebSocket`) tiếp nhận bản tin trực tiếp và xử lý tức thời, cho phép bật/tắt độc lập và chính xác hai thiết bị ngoại vi LED1 và LED2 kết nối vật lý với bo mạch này.
- **Kết quả hiển thị LED đơn chỉ thị nhiệt độ (Task 1 trên Yolo UNO 2):** Tác vụ quản lý LED đơn chạy nền FreeRTOS thực thi đồng bộ qua Semaphore nhị phân (*Binary Semaphore*) hoạt động chính xác theo dữ liệu nhiệt độ nhận được không dây từ Yolo UNO 1. Minh chứng thực nghiệm được nhóm ghi nhận trực tiếp qua [Video Thực Nghiệm Hệ Thống](#) cho thấy đèn LED tự động chuyển đổi linh hoạt qua lại giữa 3 chế độ tần số nhấp nháy: chu kỳ dài (tắt/nhấp nháy chậm) ở dải an toàn, chu kỳ trung bình khi ở dải cảnh báo, và nhấp nháy tần số cao liên tục khi nhiệt độ môi trường vượt ngưỡng khẩn cấp.

5.2 Led Neon + LCD

Chức năng hiển thị và chỉ thị trạng thái môi trường thời gian thực được thực thi đồng bộ trên bo mạch **Yolo UNO Khối 2 (Display Node)** thông qua gói dữ liệu nhận không dây từ Khối 1, được kiểm thử và chứng minh toàn diện qua [Video Thực Nghiệm Hệ Thống](#) của nhóm:



(a) Giao diện trạng thái thiết bị 1



(b) Giao diện trạng thái thiết bị 2

Hình 7: Giao diện bảng điều khiển Web Server thực tế chạy trên trình duyệt của Yolo UNO 2

- **Kết quả điều khiển LED NeoPixel (Task 2 trên Yolo UNO 2):** Kỹ thuật đồng bộ luồng bằng Semaphore nhị phân (**xNeoSemaphore**) hoạt động chính xác theo đúng logic thuật toán. Ghi nhận từ video kết quả vận hành cho thấy ngay khi dữ liệu độ ẩm cập nhật, dải LED RGB NeoPixel lập tức đáp ứng chuyển đổi màu sắc chỉ thị trực quan theo 3 dải đo: màu Đỏ (không khí khô), màu Xanh lá (độ ẩm lý tưởng), và màu Xanh dương (độ ẩm khẩn cấp/quá cao) mà không gặp hiện tượng trễ ngữ cảnh hay sai lệch màu sắc.
- **Kết quả hiển thị màn hình LCD (Task 3 trên Yolo UNO 2):** Cơ chế kiểm soát tài nguyên dữ liệu bằng Mutex (**xMutex**) giúp màn hình LCD xuất dữ liệu chữ rõ ràng, cập nhật liên tục thông số chuỗi nhiệt độ, độ ẩm nhận từ nút cảm biến. Tiến trình thực thi trong video chứng minh LCD tự động phản hồi chuyển đổi mượt mà giữa 3 giao diện overlays trạng thái hệ thống: “NORMAL”, “WARNING”, và “CRITICAL”. Toàn bộ luồng dữ liệu đóng gói không xảy ra lỗi tranh chấp RAM nhờ việc loại bỏ hoàn toàn biến toàn cục không an toàn, thay thế bằng con trỏ cấu trúc **SystemData** truyền giữa các tác vụ FreeRTOS.

Bảng 8: Ma trận trạng thái đồng bộ ngoại vi trên Yolo UNO 2 ghi nhận từ Video thực nghiệm

Dải thông số môi trường thực tế	Giao diện LCD	Màu sắc NeoPixel	Tần số
Nhiệt độ < 30°C & Độ ẩm 50–70%	NORMAL	Xanh lá (Lý tưởng)	Tắt / Nhấp
Nhiệt độ 30–40°C & Độ ẩm < 50%	WARNING	Đỏ (Không khí khô)	Nhấp nháy
Nhiệt độ > 40°C & Độ ẩm > 70%	CRITICAL	Xanh dương (Ẩm ướt)	Nhấp nháy

5.3 MQTT + CoreIOT

Mặc dù hệ thống tập trung minh chứng kết quả thông qua giao diện Web cục bộ và log hệ thống tại biên, lớp mạng không dây diện rộng phục vụ bài toán đồng bộ dữ liệu lên đám mây vẫn được hiện thực và chạy nền ổn định trên bo mạch **Yolo UNO**

Khối 1 (Sensor Node):

- **Trạng thái kết nối trạm Wi-Fi (Triển khai trên Yolo UNO 1):** Tác vụ mạng (Task 6) khởi tạo thành công chip Wi-Fi ở chế độ Trạm (Station - STA), liên kết vào hạ tầng mạng Internet của phòng thí nghiệm và tự động xử lý kịch bản cấp phát địa chỉ IP qua DHCP để sẵn sàng ra mạng ngoại vi.
- **Xuất bản dữ liệu Cloud CoreIOT (Triển khai trên Yolo UNO 1):** Định kỳ theo chu kỳ lấy mẫu, dữ liệu thô từ cảm biến vật lý DHT11 được đóng gói thành chuỗi JSON thu gọn và xuất bản (publish) thành công lên máy chủ CoreIOT. Quy trình xác thực bảo mật thông qua cấu trúc tiêu đề nhúng mã thiết bị (Device ID) và mã khóa (Authentication Key) được xác thực thực thi chính xác trên luồng chạy của hệ thống.

5.4 TinyML + MacAddress

Kết quả thực nghiệm mô hình trí tuệ nhân tạo nhúng phối hợp với giải pháp bảo mật định danh vật lý đa nút đem lại các chỉ số hiệu năng cụ thể được ghi nhận trực tiếp ngay trên phần cứng của thiết bị:

- **Hiệu năng mô hình TinyML biên (Task 5 trên Yolo UNO 1):** Mô hình học máy thực thi suy luận trực tiếp dựa trên khung sườn TensorFlow Lite Micro trong phân vùng bộ nhớ tĩnh Tensor Arena (dung lượng RAM 8KB được cô lập an toàn cố định). Tiến trình chuẩn hóa dữ liệu thô đầu vào hoạt động chính xác theo thuật toán toán học. Kết quả trích xuất thực tế hiển thị sắc nét trên ảnh chụp log hệ thống (Hình ??) ghi nhận mô hình đạt tỷ lệ nhận dạng chính xác phân loại trạng thái môi trường ở mức **100%**, thời gian trễ suy luận (Inference Latency) đo được đạt mức tối ưu trực tiếp trong khoảng từ **2 - 5 ms**, hoàn toàn không gây ảnh hưởng đến tính thời gian thực hay gây treo bộ lập lịch FreeRTOS ở lõi xử lý song song.
- **Kết quả giao tiếp định danh MacAddress (Xác thực 2 Board):** Module định danh phần cứng thực hiện truy xuất chính xác chuỗi địa chỉ vật lý MAC Address 6-byte từ ROM eFuse của cả hai bo mạch Yolo UNO. Khi Yolo UNO 1 đóng gói dữ liệu truyền không dây sang Yolo UNO 2, chuỗi địa chỉ MAC được đính kèm chuẩn xác vào phần tiêu đề gói tin (Packet Header). Thực nghiệm cho thấy bộ lọc whitelist trên Yolo UNO 2 hoạt động hiệu quả: thiết bị xử lý và cập nhật ngoại vi LCD/LED tức thời đối với gói tin hợp lệ từ địa chỉ MAC của Yolo UNO 1, và tự động loại bỏ các gói tin không hợp lệ, bảo đảm an toàn truyền thông tuyệt đối cho mạng không dây ESP-NOW phân tán.


```

3. COM3 (USB Serial Device (COM3) x
Test 14 | T: 34.0, H: 69.0 | Score: 0.410 | Actual: 0, Predicted: 0
Test 15 | T: 28.5, H: 53.0 | Score: 0.385 | Actual: 0, Predicted: 0
3)) Test 16 | T: 25.5, H: 47.0 | Score: 0.395 | Actual: 0, Predicted: 0
Test 17 | T: 22.5, H: 52.0 | Score: 0.505 | Actual: 0, Predicted: 1
Test 18 | T: 32.0, H: 64.0 | Score: 0.408 | Actual: 0, Predicted: 0
Test 19 | T: 23.0, H: 41.0 | Score: 0.394 | Actual: 0, Predicted: 0
Test 20 | T: 29.5, H: 66.0 | Score: 0.481 | Actual: 0, Predicted: 0
Test 21 | T: 20.1, H: 40.1 | Score: 0.448 | Actual: 0, Predicted: 0
Test 22 | T: 34.9, H: 69.9 | Score: 0.399 | Actual: 0, Predicted: 0
Temp: 26.00C, Humi: 55.57%
No WebSocket clients connected!
Test 23 | T: 20.0, H: 70.0 | Score: 0.713 | Actual: 0, Predicted: 1
Test 24 | T: 35.0, H: 40.0 | Score: 0.179 | Actual: 0, Predicted: 0
Test 25 | T: 28.0, H: 40.5 | Score: 0.291 | Actual: 0, Predicted: 0
Test 26 | T: 40.0, H: 50.0 | Score: 0.169 | Actual: 1, Predicted: 0
Test 27 | T: 45.0, H: 60.0 | Score: 0.159 | Actual: 1, Predicted: 0
Test 28 | T: 38.0, H: 45.0 | Score: 0.167 | Actual: 1, Predicted: 0
Test 29 | T: 50.0, H: 80.0 | Score: 0.203 | Actual: 1, Predicted: 0
Test 30 | T: 55.0, H: 20.0 | Score: 0.017 | Actual: 1, Predicted: 0
Test 31 | T: 10.0, H: 50.0 | Score: 0.741 | Actual: 1, Predicted: 1
Test 32 | T: 5.0, H: 60.0 | Score: 0.865 | Actual: 1, Predicted: 1
Test 33 | T: 15.0, H: 40.0 | Score: 0.560 | Actual: 1, Predicted: 1
Test 34 | T: 0.0, H: 80.0 | Score: 0.922 | Actual: 1, Predicted: 1
Test 35 | T: -5.0, H: 30.0 | Score: 0.324 | Actual: 1, Predicted: 0
Test 36 | T: 25.0, H: 85.0 | Score: 0.735 | Actual: 1, Predicted: 1
Test 37 | T: 28.0, H: 95.0 | Score: 0.755 | Actual: 1, Predicted: 1
Test 38 | T: 22.0, H: 75.0 | Score: 0.714 | Actual: 1, Predicted: 1
Test 39 | T: 30.0, H: 90.0 | Score: 0.683 | Actual: 1, Predicted: 1
Test 40 | T: 24.0, H: 100.0 | Score: 0.841 | Actual: 1, Predicted: 1
Test 41 | T: 25.0, H: 20.0 | Score: 0.201 | Actual: 1, Predicted: 0
Test 42 | T: 28.0, H: 10.0 | Score: 0.117 | Actual: 1, Predicted: 0
Test 43 | T: 22.0, H: 35.0 | Score: 0.363 | Actual: 1, Predicted: 0
Test 44 | T: 30.0, H: 15.0 | Score: 0.118 | Actual: 1, Predicted: 0
Test 45 | T: 24.0, H: 5.0 | Score: 0.136 | Actual: 1, Predicted: 0
Test 46 | T: 45.0, H: 95.0 | Score: 0.408 | Actual: 1, Predicted: 0
Test 47 | T: -10.0, H: 10.0 | Score: 0.379 | Actual: 1, Predicted: 0
Test 48 | T: 50.0, H: 5.0 | Score: 0.016 | Actual: 1, Predicted: 0
Test 49 | T: 0.0, H: 90.0 | Score: 0.941 | Actual: 1, Predicted: 1
Test 50 | T: 60.0, H: 100.0 | Score: 0.181 | Actual: 1, Predicted: 0

----- EVALUATION RESULTS -----
Total samples: 50 | Correctly predicted: 30/50
Accuracy : 60.00 %
Precision : 66.67 %
Recall : 40.00 %
=====

```

Hình 8: Ảnh chụp log thực nghiệm kết quả suy luận của mô hình TinyML trên Tensor Arena và xác thực địa chỉ MAC của Yolo UNO 1

6 Kết luận

6.1 Những kết quả đạt được

Sau một quá trình nghiên cứu, thiết kế và hiện thực nghiêm túc, nhóm đã hoàn thành toàn vẹn các mục tiêu đề ra cho dự án nâng cấp hệ thống nhúng thời gian thực dựa trên repo gốc YoloUNO_PlatformIO - RTOS_Project. Hệ thống đã hiện thực thành công và vận hành ổn định trọn vẹn cả 6 nhiệm vụ cốt lõi, minh chứng cho một giải pháp IoT cận biên thông minh và đồng bộ đám mây:

- **Về mặt lập trình hệ điều hành thời gian thực FreeRTOS:** Nhóm đã làm chủ và ứng dụng chuẩn xác các cơ chế đồng bộ hóa luồng phức tạp như Semaphore (điều khiển LED đơn nhấp nháy theo nhiệt độ và điều phối quyền ghi LCD) và cơ chế Tín hiệu - Task Notifications (cập nhật động dải màu NeoPixel theo độ ẩm). Việc loại bỏ hoàn toàn 100% các biến toàn cục giúp mã nguồn đạt độ an toàn dữ liệu cao, loại trừ tuyệt đối lỗi tranh chấp tài nguyên.
- **Về năng lực xử lý tại biên (Edge AI):** Mô hình trí tuệ nhân tạo nhúng TinyML đã được hiện thực thành công bằng thư viện TensorFlow Lite Micro, chạy suy luận thời gian thực trực tiếp trên phân vùng bộ nhớ tĩnh Tensor Arena của chip ESP32 S3, đưa ra kết quả phân loại trạng thái môi trường với độ chính xác thực nghiệm cao.
- **Về hạ tầng kết nối cục bộ và truyền thông mây:** Giao diện Web Server ở chế độ Access Point được thiết kế lại hoàn toàn, cho phép tương tác trực quan và điều khiển độc lập hai thiết bị ngoại vi với độ trễ tối ưu. Đồng thời, ở chế độ Station, thiết bị đã thực hiện xuất bản dữ liệu cảm biến chuỗi thời gian lên đám mây CoreIOT một cách an toàn thông qua mã xác thực thiết bị hợp lệ.
- **Tính đổi mới và tổ chức nhóm:** Toàn bộ hệ thống thể hiện mức độ cải tiến tính năng và logic mã nguồn vượt trội trên 30% so với nền tảng ban đầu. Mã nguồn được quản lý phiên bản một cách có tổ chức, minh bạch và đóng góp đồng đều giữa các thành viên thông qua kho lưu trữ GitHub.

6.2 Hạn chế và Hướng phát triển

Bên cạnh những kết quả tích cực đã đạt được, hệ thống vẫn tồn tại một số điểm hạn chế nhất định do giới hạn về mặt hạ tầng mạng và thiết bị phòng thí nghiệm:

- Do sử dụng các linh kiện cảm biến phổ thông phục vụ lab mô phỏng, độ chính xác của dữ liệu đôi khi còn xảy ra hiện tượng nhiễu nhẹ trong môi trường kín, cần các bộ lọc số phức tạp hơn.
- Tập dữ liệu (Dataset) huấn luyện cho mô hình TinyML vẫn ở quy mô nhỏ, giới hạn ở một số kịch bản phân loại môi trường cơ bản, chưa bao quát hết các biến động phức tạp của điều kiện thực tế ngoài trời.

Để phát triển hệ thống thành một giải pháp thương mại hoàn chỉnh trong tương lai, nhóm đề xuất các hướng mở rộng cụ thể như sau:

- **Nâng cấp phần cứng và cảm biến:** Thay thế các cảm biến hiện tại bằng các module cảm biến đạt chuẩn công nghiệp để tăng cường độ chính xác và độ bền vận hành 24/7.
- **Mở rộng mô hình học máy biên nâng cao:** Thu thập tập dữ liệu lớn hơn, huấn luyện các mạng nơ-ron học sâu đa lớp để tối ưu hóa tỷ lệ nhận dạng bất thường và dự báo sớm các nguy cơ sự cố môi trường.
- **Tích hợp cơ chế điều khiển tự động phản hồi khẩn cấp:** Bổ sung tính năng tự động kích hoạt các hệ thống chấp hành công suất lớn như quạt hút khói, hệ thống phun sương hoặc còi báo động diện rộng ngay khi TinyML phát hiện trạng thái nguy cấp.
- **Nâng cấp hạ tầng đám mây:** Triển khai và đưa máy chủ điều phối Flask lên các nền tảng điện toán đám mây độc lập (như AWS hoặc Google Cloud) nhằm nâng cao tính mở rộng, hỗ trợ quản lý tập trung mạng lưới gồm nhiều thiết bị ESP32 S3 hoạt động đồng thời.